Dashgen: сборка дашбордов Qlik Sense из Excel с ИИ

Проектная документация, версия 1.0, 3 ноября 2026

1. Назначение документа

Документ описывает проект целиком: цель, среду, пройденный путь, архитектуру, план разработки и каждый файл проекта с примерами. Он написан так, чтобы с проектом мог работать другой разработчик или ИИ-модель без доступа к прежней переписке. Правила работы с проектом собраны в разделе 16, полный код рабочего скрипта в приложении А.

2. Цель и контекст

Цель: по Excel-файлу и короткому описанию отчёта приложение создаёт в Qlik Sense (on-premise) приложение с моделью данных, мастер-элементами и листами. Готовый файл .qvf выгружается из Hub или QMC (готовый .qvf вне движка Qlik собрать нельзя, это закрытый бинарный формат).

Среда:

Компонент	Значение
Рабочая машина	Mac, терминал zsh, Node.js v26.13.0, папка проекта ~/qlik_build
Qlik	Sense Enterprise on Windows (on-premise), Qlik Cloud не используется
Сервер	dc3-qliks-ap-03.onyx.local, Engine: порт 4747 (проверено командой nc, ответ succeeded)
QRS API	порт 4242 (пока не проверялся)
Аутентификация	клиентские сертификаты из QMC (имя машины Mac-dev) и заголовок X-Qlik-User
Библиотеки Node	enigma.js, ws
Схема Engine	enigma.js/schemas/12.612.0.json
Версия свойств объектов	1.39.8 (версия клиента Qlik на сервере)

Что работает сейчас. Одна команда `node build_dashboard_v2.js` создаёт приложение TEST_Dashboard_Inline: inline-данные (600 заказов), 3 мастер-измерения, 4 мастер-меры, лист «Обзор» с тремя фильтрами, четырьмя KPI, линейным графиком, столбчатой диаграммой и таблицей. Результат проверен в Hub.

3. Принципы

1. **ИИ решает, код исполняет.** Модель возвращает только описание дашборда (spec.json) по строгой схеме. Объекты Qlik, скрипт загрузки и раскладку создаёт проверенный код.
2. **Данные остаются внутри.** Excel читается локально и загружается на внутренний сервер Qlik. В ИИ уходит только безопасный профиль без значений.
3. **Человек утверждает.** Перед отправкой в ИИ и перед сборкой пользователь видит и подтверждает результат.
4. **Источники подключаются адаптерами.** Excel первый, затем CSV, базы данных, QVD.
5. **Воспроизводимость.** Одна команда, журнал действий, код в GitLab без сертификатов и ключей.

4. Установка окружения (повторяемая инструкция)

Шаг 1. Node.js LTS. Установщик .pkg с nodejs.org (или brew install node). Проверка: `node -v`, `npm -v`.

Шаг 2. Папка и библиотеки.

```
mkdir -p ~/qlik_build && cd ~/qlik_build
npm install enigma.js ws
```

Шаг 3. Сертификаты. В QMC: Certificates, указать Machine name (например, Mac-dev), включить Include secret key, нажать Export certificates. Получить архив с файлами root.pem, client.pem, client_key.pem (server.pem и server_key.pem не нужны). Распаковать:

```
mkdir -p ~/qlik_build/certs
unzip -j ~/Downloads/Mac-dev.zip -d ~/qlik_build/certs
chmod 600 ~/qlik_build/certs/client_key.pem
```

Файлы ключей нельзя пересылать, коммитить в git и вставлять в чат.

Шаг 4. Проверка сети.

```
nc -vz dc3-qliks-ap-03.onyx.local 4747
```

Нужен ответ succeeded. Если таймаут, проверьте VPN или корпоративную сеть.

Шаг 5. Настройки. В блоке CONFIG файла build_test_dashboard.js задаются host, port, certDir, userDirectory (домен из Hub, вид ДОМЕН и логин), userId, appName, rejectUnauthorized. Правка делается в nano или другом редакторе простого текста (TextEdit портит кавычки).

Шаг 6. Проверка синтаксиса и запуск.

```
node --check build_dashboard_v2.js
node build_dashboard_v2.js
```

Успех: строки «Приложение создано», «Данные загружены», «Готово». Приложение появляется в Hub, раздел Work.

Как создавать файлы в терминале. Файл, который нельзя скачать, создаётся блоком `cat > файл << 'EOF'` … `EOF`. Блок вставляется целиком и отдельно от других команд, строка EOF стоит на отдельной строке без лишних символов. Остальные команды выполняются по одной.

5. Хронология и найденные проблемы

Проблема	Причина	Решение
Скрипт не скачивался из чата	Карточка файла недоступна	Создание файла блоком cat с EOF
SyntaxError в CONFIG	Пропущена запятая, умные кавычки TextEdit	Правка в nano, проверка node --check
Cannot find module …jsnpm	Две команды вставлены слитно	Команды вставлять по одной
Файл записался дважды, EOF слипся с командой	Блок и следующие команды вставлены вместе	Обрезка awk, затем команды запуска отдельно
Панель фильтров: незавершённая визуализация	У панели не объявлен список дочерних объектов	Три отдельных листбокса вместо панели

Проблема	Причина	Решение
Графики и таблица пустые при наличии данных	Клиент Qlik добавляет к объекту около 60 служебных свойств (цвета, оси, легенда, версия), через Engine API они не появляются	Сравнение объекта из API и его копии из интерфейса, свойства по умолчанию зашиты в сборщик

Диагностика getLayout показала, что у объектов нет ошибок и данные приходят. Значит, причина была именно в отсутствующих свойствах отображения.

6. Архитектура целевого приложения

Конвейер модулей. Граница безопасности проходит между модулями privacy и ai: до неё всё локально.

№	Модуль	Назначение	Где работает
1	excel-reader	Чтение листов, заголовков, строк	локально
2	profiler	Типы колонок, уникальные значения, пропуски, кандидаты в ключи и связи	локально
3	privacy	Безопасный профиль по выбранному режиму, dry run, журнал отправок	локально
4	ai	Роли колонок, проект дашборда, разбор ошибок, документация	облако или локальная модель
5	spec-validator	Проверка spec.json по схеме, существование полей, разрешённые типы объектов	локально
6	qvs-generator	Скрипт загрузки по спецификации: типы, даты, календарь, связи	локально
7	delivery	Доставка Excel в Qlik	внутренняя сеть
8	builder	Создание приложения и объектов, библиотека шаблонов, раскладка	внутренняя сеть

№	Модуль	Назначение	Где работает
9	verify	Диагностика объектов, сверка итогов с Excel, проверка выражений движком	внутренняя сеть
10	docs	Описание отчёта для Confluence	ИИ

7. Доставка Excel в Qlik

- **Вариант А (основной): data connection на папку.** Скрипт загрузки читает lib://ИМЯ_ПОДКЛЮЧЕНИЯ/файл.xlsx. Папка на сервере или в сетевом хранилище, у служебной учётной записи Qlik есть доступ на чтение. Подходит для обновления по расписанию.
- **Вариант Б: загрузка через QRS API** (порт 4242, те же сертификаты). Нужен, если нет доступа к папке.
- **Вариант В: inline-данные** в скрипте. Только для небольших таблиц и тестов (так сделан текущий тестовый дашборд).

8. Приватность

Любой облачный ИИ получает всё, что ему отправили, поэтому защита строится на том, что именно уходит наружу.

Режим	Что уходит в ИИ	Когда использовать
schema (по умолчанию)	Имена листов и колонок, типы, число строк и уникальных значений. Значений нет	Основной режим
synthetic	То же и выдуманные значения нужного типа	Когда ИИ нужны примеры форматов
tokenized	Имена заменены на TABLE_A, COL_3, значений нет. Таблица соответствия хранится только локально	Если названия колонок чувствительны
local	Ничего не покидает контур, используется локальная модель	Если политика запрещает внешние сервисы

Правила.

- В модуль ai технически передаётся только вывод модуля privacy, а не данные из excel-reader.
- Список запрещённых колонок в конфиге (ФИО, ИНН, телефоны, контрагенты и т. п.). Они исключаются из профиля полностью. Дополнительно автоопределение по названиям и регулярным выражениям.
- Свободный текст, комментарии в ячейках и метаданные файла (автор, пути) не отправляются никогда.
- Режим dry run включён по умолчанию: пользователь видит итоговый JSON и подтверждает отправку.
- Журнал отправок (время, содержимое, адресат) хранится локально.
- Ключи API в переменных окружения. Сертификаты Qlik, ключи и журналы вне git, права chmod 600.
- Описание отчёта, которое пользователь пишет для ИИ, не содержит реальных цифр и имён клиентов.
- До первого запуска на реальных данных режимы согласуются со службой безопасности.

9. Роль ИИ

Задача	Исполнитель
Чтение Excel, типы, пропуски, связи	код
Роли колонок (мера, измерение, дата, ключ), подписи, форматы	ИИ по безопасному профилю
Выбор показателей и срезов по описанию отчёта	ИИ
Формулы мер и set analysis	ИИ, затем проверка движком (CheckExpression)
Типы визуализаций и состав листов	ИИ из разрешённого списка
Раскладка по сетке 24 на 12	код
Скрипт загрузки и свойства объектов	код по шаблонам
Разбор ошибок загрузки и пустых объектов	ИИ предлагает правку spec.json
Правки словами	ИИ формирует патч spec.json

Задача	Исполнитель
Описание для Confluence	ИИ

Технически. Единый интерфейс `provider.complete({ system, prompt, schema })`. Провайдеры подключаются отдельными файлами (Claude API, OpenAI-совместимые API, локальная модель, например через Ollama). Температура ноль, ответ строго по JSON-схеме (иначе отбрасывается и запрос повторяется), кэш по хешу входа, журнал запросов.

Контур самоисправления. Ошибка загрузки или объекта передаётся модели, она предлагает правку spec.json, код применяет её и пересобирает. Лимит 2-3 попытки, затем остановка и отчёт человеку.

10. Этапы разработки и критерии готовности

Этап	Что делаем	Критерий готовности
0. Подготовка	Согласовать режимы приватности, выбрать ИИ-провайдера, подготовить обезличенный тестовый Excel, выбрать способ доставки, завести репозиторий GitLab с .gitignore	Решения зафиксированы письменно
1. Ядро сборки	Перенести build_dashboard_v2.js в модули builder и templates, сборка из spec.json	Тестовый дашборд собирается из файла spec.json
2. Чтение и профиль	excel-reader, profiler	Таблица «колонка, тип, роль» по тестовому Excel
3. Приватность	privacy: режим schema, запрещённые колонки, dry run, журнал	JSON без значений, пользователь подтверждает отправку
4. Генератор загрузки	qvs-generator, delivery	Загрузка без ошибок и без синтетических ключей

Этап	Что делаем	Критерий готовности
5. ИИ-семантика и проект	Интерфейс провайдера, шаблоны запросов, схема ответа, spec-validator	spec.json по описанию проходит схему и собирается
6. Проверка качества	verify: ошибки объектов, сверка сумм и числа строк с Excel, проверка выражений	Отчёт о расхождениях в консоли
7. Самоисправление	Передача ошибок модели, применение патчей, лимит попыток	Типовые ошибки исправляются без пользователя
8. Диалог правок и документация	Правки spec.json словами, описание для Confluence	Правка словами меняет лист
9. Упаковка	README, примеры, запуск `node dashgen.js файл.xlsx`, экспорт .qvf через QRS, публикация в стрим	Запуск на новом файле без правок кода

Этапы 0-4 дают сборку без ИИ (спецификацию пишет человек), этапы 5-7 добавляют ИИ. Работоспособность проверяется поэтапно.

11. Риски и меры

Риск	Мера
Обновление клиента Qlik меняет свойства по умолчанию (сейчас 1.39.8)	Пересоздать шаблоны после обновления, хранить версию в конфиге
Даты и числа из Excel (серийные номера, десятичный разделитель)	Явные форматы в скрипте загрузки, тесты
Одинаковые названия колонок на разных листах дают синтетические ключи	Переименование полей, предупреждение
ИИ ошибается в ролях колонок и формулах	Утверждение spec.json, сверка с Excel, проверка выражений движком
Утечка данных	Режимы приватности, dry run, журнал, запрет значений и свободного текста

Риск	Мера
Большие файлы	QVD-слой, Excel как первичный источник только для небольших данных
Нестандартные объекты (расширения Uni)	Шаблон берётся один раз из рабочего объекта в библиотеку шаблонов
Ручной обмен командами и выводом	Разработка в Claude Code на Mac, где агент сам запускает команды

12. Что нужно решить до старта

1. Режим приватности на старте и согласование с безопасностью.
2. ИИ-провайдер: облачный (какой), корпоративный или локальная модель.
3. Доставка Excel: data connection на папку или QRS API (порт 4242).
4. Обезличенный тестовый Excel (20-50 строк на лист) и описание связей между листами.
5. Описание отчёта: показатели, срезы, графики, без реальных цифр.
6. Куда публиковать: личная область Work или стрим.
7. Возможные изменения сервера Qlik (при наличии прав): папка и data connection, открытие порта 4242, отдельная служебная учётная запись с минимальными правами, тестовый стрим, сертификат с нужным именем сервера (тогда rejectUnauthorized можно включить), отдельный тестовый сервер. Изменения согласуются с администратором Qlik и службой безопасности.

13. Структура проекта

Текущее состояние папки ~/qlik_build:

```
qlik_build/
  build_test_dashboard.js   первая версия сборщика; из него v2 берёт CONFIG и
LOAD_SCRIPT
  build_dashboard_v2.js     рабочий сборщик дашборда (главный файл)
  diagnose_dashboard.js     проверка объектов приложения
  compare_objects.js        сравнение объекта из API и его копии из
интерфейса
  heal_objects.js           дополнение объектов свойствами по умолчанию из
копий
  templates.json            сохранённые свойства по умолчанию (создаётся
heal_objects.js)
  certs/                    root.pem, client.pem, client_key.pem (+
server.pem, server_key.pem)
  node_modules/, package.json, package-lock.json
```

Целевая структура приложения dashgen:

```
dashgen/
  config/
    config.example.json     пример настроек (реальный config.json не в git)
  certs/                    не в git
  src/
    excel-reader.js
    profiler.js
    privacy/                schema.js, synthetic.js, tokenized.js, dryrun.js,
sendlog.js
    ai/                     provider.js, providers/, prompts/, schemas/
    spec-validator.js
    qvs-generator.js
    delivery/               data-connection.js, qrs-upload.js, inline.js
    builder.js
    templates/              kpi.js, bar.js, line.js, table.js, listbox.js
    layout.js
    verify.js
    docs.js
  specs/                    готовые spec.json
  logs/                     не в git
  dashgen.js                точка входа
  package.json, README.md, .gitignore, .env.example
```

14. Текущие файлы: описание и примеры

Файл	Роль	Статус
build_test_dashboard.js	Первая версия сборщика, источник CONFIG и LOAD_SCRIPT	рабочий, нужен v2
build_dashboard_v2.js	Сборщик тестового дашборда со свойствами по умолчанию	основной
diagnose_dashboard.js	Проверка объектов листа через getLayout	вспомогательный
compare_objects.js	Сравнение свойств двух столбчатых диаграмм	разовый, для отладки
heal_objects.js	Дополнение объектов свойствами из копий, созданных в интерфейсе	устарел после v2
templates.json	Выгрузка шаблонов (создаёт heal_objects.js)	справочный
certs/*	Клиентские сертификаты Qlik	секретные, не в git

14.1. build_test_dashboard.js

Роль. Первый сборщик. Сегодня важен как хранилище двух блоков, которые build_dashboard_v2.js вычитывает регулярными выражениями: `const CONFIG = {...};` и `const LOAD_SCRIPT = ... ;`. Эти два фрагмента нельзя переименовывать и менять их формат записи.

CONFIG (пример, значения подставляются свои):

```
const CONFIG = {
  host: 'dc3-qliks-ap-03.onyx.local',
  port: 4747,
  certDir: './certs',
  userDirectory: 'ДОМЕН',     // из Hub: ДОМЕН и логин
  userId: 'ЛОГИН',
  appName: 'TEST_Dashboard_Inline',
  rejectUnauthorized: false   // true, если имя сервера есть в сертификате
};
```

LOAD_SCRIPT. Скрипт загрузки Qlik в шаблонной строке. Содержит два inline-справочника (Regions: Region, Manager; Products: Product, Category, Price) и таблицу фактов Sales на 600 случайных заказов за 2025 год (OrderID, Date, Month, Year,

Region, Product, Qty, Amount). Amount считается как Qty, умноженное на цену из справочника через Lookup. В строке не должно быть символа обратной кавычки.

14.2. build_dashboard_v2.js

Роль. Главный сборщик: один запуск создаёт приложение и лист целиком. Полный код приведён в приложении А.

Запуск: `node build_dashboard_v2.js`

Порядок действий:

1. Читает CONFIG и LOAD_SCRIPT из build_test_dashboard.js.
2. Подключается к Engine по WebSocket (wss, порт 4747) с клиентскими сертификатами и заголовком X-Qlik-User.
3. Удаляет приложения с тем же именем (повторяемый запуск).
4. createApp, openDoc, setScript, doReload (при ошибке загрузки останавливается).
5. Создаёт 3 мастер-измерения (Регион, Товар, Месяц) и 4 мастер-меры (Выручка, Заказы, Средний чек, Количество).
6. Создаёт объекты: 4 KPI, линейный график, столбчатая диаграмма, таблица, 3 листбокса (Регион, Товар, Год).
7. Создаёт лист «Обзор» (сетка 24 колонки на 12 строк), doSave.

Ключевые константы и функции:

- `VER = '1.39.8'` версия свойств объектов клиента Qlik на сервере.
- `BASE`, `CHART`, `BAR_DEFAULTS`, `LINE_DEFAULTS`, `TABLE_DEFAULTS` свойства по умолчанию, которые клиент Qlik добавляет сам. Без них графики и таблица создаются, но не отображаются.
- `withDefaults(base, props)` копирует свойства по умолчанию и накладывает свойства объекта.
- `cell(name, type, col, row, colspan, rowspan)` описывает ячейку листа, вычисляет `bounds` в процентах.
- Меры используются через `qLibraryId` (мастер-меры), измерения заданы напрямую по полю: `{ qDef: { qFieldDefs: ['Region'], qFieldLabels: ['Регион'] } }`.
- Панель фильтров (filterpane) не используется: дочерние листбоксы не отображались. Вместо неё три отдельных листбокса.

Раскладка листа (24 колонки, 12 строк):

Объект	col	row	colspan	rowspan
Листбоксы (3 шт.)	0	0, 4, 8	4	4
KPI (4 шт.)	4, 9, 14, 19	0	5	3
Линейный график	4	3	10	4
Столбчатая диаграмма	14	3	10	4
Таблица	4	7	20	5

Пример вывода при успехе:

```
Удалено старое приложение 5d988aaf-ea50-44ed-8eb4-626f8182e66a
Приложение создано: 8f1c...
Данные загружены
Готово. Приложение «TEST_Dashboard_Inline» (ID 8f1c...) в Hub → Work.
```

14.3. diagnose_dashboard.js

Роль. Открывает приложение с именем из CONFIG, обходит ячейки листов, для каждого объекта читает `getLayout()` и печатает тип, заголовок и статус. Ошибки гиперкуба и листбокса выводятся явно. Настройки берёт из build_test_dashboard.js.

Запуск: `node diagnose_dashboard.js`

Пример вывода (реальный запуск):

```
Листов: 1

Лист: Обзор | ячеек: 8
  filterpane                                  OK
  kpi          Выручка                        OK (строк: 1, колонок: 1)
  linechart    Выручка по месяцам             OK (строк: 12, колонок: 2)
  barchart     Выручка по регионам            OK (строк: 4, колонок: 2)
  table        Детализация: товар и регион    OK (строк: 20, колонок: 5)
```

Важный вывод из отладки: статус OK в диагностике означает, что движок отдаёт данные, но не гарантирует, что объект рисуется. Отсутствие служебных свойств отображения диагностика не ловит.

14.4. compare_objects.js

Роль. Находит в приложении две столбчатые диаграммы (созданную скриптом, у неё id длиной 36 символов, и копию из интерфейса, id короткий) и печатает три списка: свойства только в копии, отличающиеся свойства, свойства только в оригинале. Сохраняет `bar_original.json` и `bar_copy.json`.

Пример вывода (фрагмент):

```
Найдено столбчатых диаграмм: 2 [ 'fqwNmq', '3649509c-4819-4d30-af66-
0e74b72772d6' ]

=== ТОЛЬКО В КОПИИ (работает) ===
color.auto = true
color.mode = "primary"
legend.show = true
dimensionAxis.maxVisibleItems = 10
version = "1.39.8"
...
=== ОТЛИЧАЮТСЯ ===
=== ТОЛЬКО В ОРИГИНАЛЕ ===
```

Именно этот вывод показал, что причиной пустых графиков были недостающие свойства отображения.

14.5. heal_objects.js

Роль. Для типов barchart, linechart, table берёт копию объекта, созданную в интерфейсе (короткий id), и дополняет объекты скрипта (id из 36 символов) недостающими свойствами через `setProperties`. Не перезаписывает существующие поля, пропускает `qInfo`, `qMetaDef`, `qHyperCubeDef`, `qListObjectDef`. Сохраняет шаблоны в templates.json. С выходом build_dashboard_v2.js больше не нужен.

14.6. templates.json

Словарь `{ "barchart": {...свойства объекта...}, ... }`. Содержит полные свойства работающих объектов из интерфейса, годится как источник для новых шаблонов (например, для pivot-table или pie).

14.7. certs/

Файл	Назначение	Нужен скрипту
root.pem	корневой сертификат (параметр ca)	да
client.pem	клиентский сертификат (cert)	да
client_key.pem	приватный ключ клиента (key)	да, chmod 600
server.pem, server_key.pem	серверные сертификаты	нет

Секретные файлы никогда не пересылаются, не вставляются в чат и не попадают в git.

14.8. package.json

```
{
  "name": "qlik_build",
  "dependencies": { "enigma.js": "*", "ws": "*" }
}
```

Зависимости: enigma.js (клиент Engine API), ws (WebSocket для Node.js). Для будущих модулей добавятся exceljs или xlsx (чтение Excel) и ajv (проверка JSON-схемы).